

MACHINE LEARNING ENGINEERING

FASE 4 | AULA 03 -

BERT

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	15
REFERÊNCIAS.....	16

O QUE VEM POR AÍ?

Imagine um mundo em que máquinas compreendem e respondem a textos humanos com uma precisão quase perfeita. Esse futuro já é realidade graças ao BERT, uma inovação baseada nos poderosos transformers.

Nessa aula, vamos explorar como o BERT está transformando o campo do Processamento de Linguagem Natural, proporcionando uma compreensão contextual profunda que está revolucionando aplicações em diversas áreas. Vamos desvendar o poder dos transformers juntos?

HANDS ON

A essa altura você já deve ter ouvido falar dos maravilhosos modelos generativos, como os da família GPT, Gemini, LLAMA e outros, e se maravilhado com as infinitas possibilidades que eles oferecem.

Isso tudo foi possível devido a várias descobertas em redes neurais profundas. Essa revolução vai muito além da geração de texto e atinge todas as áreas do Processamento de Linguagem Natural, como classificação de texto, sumarização, tradução, perguntas e respostas, chatbots, compreensão de linguagem natural e muito mais.

Nessa aula, você vai conhecer o BERT (Bidirectional Encoder Representations from Transformers); também iremos falar sobre a revolucionária arquitetura Transformers, que vem sendo usada como base em vários dos modelos estado-da-arte que temos atualmente.

SAIBA MAIS

Atualmente vários modelos e arquiteturas diferentes são lançados praticamente todas as semanas; a inovação nunca foi tão rápida no campo de aprendizado de máquina. Isso se deve principalmente a alguns componentes:

- **Arquitetura transformers e a camada de atenção** proposta em 2017 em um artigo publicado pelo time de pesquisadores da Google, “Attention is all you need”. Essa arquitetura está presente em vários dos modelos mais famosos, como os da família GPT (Generative Pretrained Transformer), Github Copilot, BERT (Bidirectional Encoder Representations from Transformers) e outros.
- **Modelos pré-treinados em tarefas e base de dados genéricos.** Em tarefas de processamento de imagens, essa técnica já era utilizada desde o início dos anos 2010, mas em PNL os artigos que propunham técnicas de utilização de modelos pré-treinados começaram a aparecer pelo ano de 2018.
- **Hubs de modelos que ajudam a centralizar esses modelos e abstrair sua complexidade de implementação**, padronizando e facilitando sua utilização. O maior exemplo desses hubs é o [Hugging Face](#). Nele, é possível explorar e baixar modelos estado-da-arte, filtrar por tarefas, linguagens e outras variáveis, além de facilitar seu treinamento, suportando tanto Tensorflow como Pytorch através de uma biblioteca open-source. Além disso, ele possui uma excelente documentação, mostrando como treinar e avaliar os modelos; você inclusive tem acesso a dados para lhe auxiliar no desenvolvimento dos modelos.

Nesta aula vamos estudar o modelo pré-treinado BERT.

BERT (Bidirectional Encoder Representations from Transformers)

Conforme o próprio nome já diz, BERT é um modelo de linguagem natural baseado em Transformers desenvolvido pelo Google. Lançado em 2018, ele introduziu uma abordagem bidirecional para o treinamento de modelos de linguagem, permitindo uma melhor compreensão do contexto das palavras em uma frase.

Antes do BERT, a maioria dos modelos de linguagem processava texto de forma unidirecional, da esquerda para a direita ou vice-versa. O BERT, no entanto, considera o contexto completo de uma palavra ao olhar para os dois lados da sequência. Isso permite uma compreensão mais rica e precisa das palavras e suas relações contextuais.

Como o BERT funciona?

O BERT é baseado na arquitetura transformers, que é uma arquitetura excelente em capturar sequências longas de dados e lidar com grande quantidade de dados. Ela superou as redes neurais recorrentes (RNN) tanto em termos de qualidade de previsões como nos custos de treinamento. Vamos entender mais a fundo essa arquitetura poderosa?

Redes neurais recorrentes ainda são amplamente usadas em PNL até hoje, principalmente em arquiteturas Encoder-Decoder. A fraqueza dessa arquitetura está na criação de um encoder que represente o significado da sequência de variáveis de entrada inteira, visto que será tudo que o decoder terá acesso. Desenvolver bons modelos com essa arquitetura é especialmente desafiador quando tratamos de sequências longas, em que grande parte da informação pode ser perdida durante a compressão até uma única representação fixa.

Os transformers também consistem em codificadores (encoders) e decodificadores (decoders). O encoder converterá uma sequência de tokens em uma sequência de vetores embeddings, que são comumente chamados de hidden state ou contexto. Já o decoder usará o resultado do encoder para gerar uma sequência de tokens, um por vez.

Em contrapartida, o BERT é um modelo da classe Encoder-Only, visto que usa apenas os codificadores (encoder). Ele foi pré-treinado usando o BookCorpus e a Wikipedia em língua inglesa.

Os embeddings usados no BERT são divididos em três, conforme a figura 1: token, segmentação e posição, cada um passando uma informação diferente e complementar para que o modelo consiga aprender padrões, relações e semântica. Perceba que há tokens especiais, como o token [CLS], que indica o início da sequência, e o [SEP], que indica separação entre a sequência, como pontuação, por exemplo.

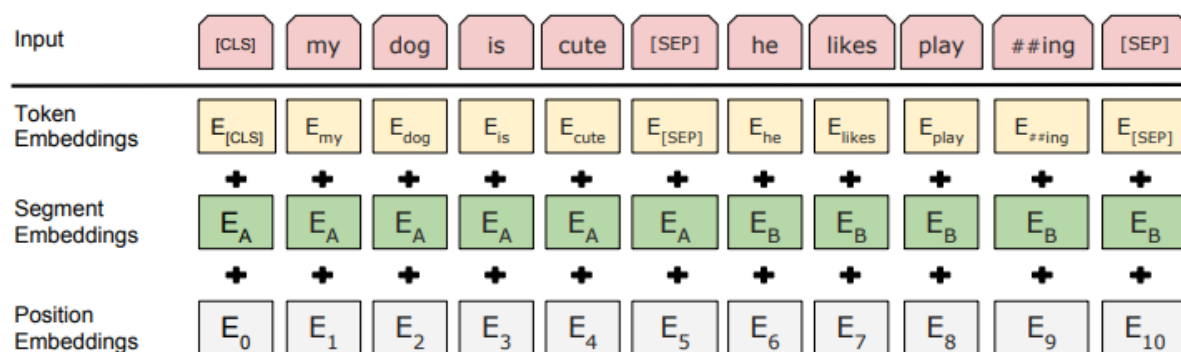


Figura 1 - BERT - Representação das variáveis de entrada

Fonte: Devlin et al. (2019)

- **Token embedding:** cada token é mapeado para um vetor multidimensional que captura a semântica e o contexto. No BERT, esse vetor tem dimensão 768.

Ex:

Token: [Eu] [amo] [NLP]

Embedding: [v1] [v2] [v3]

- **Embedding de Segmentação:** é usado para distinguir segmentos diferentes em tarefas que envolvam pares de entrada, como classificação ou perguntas e respostas. Dessa forma, há um embedding que indica a qual sentença aquele token pertence.

Ex.

Token: [Eu] [amo] [NLP] [É] [muito] [interessante]

Segmento: [A] [A] [A] [B] [B] [B]

Embedding: [sA] [sA] [sA] [sB] [sB] [sB]

- **Embedding de Posição:** captura a posição de cada token na sentença de entrada. Isso é necessário porque a arquitetura transformer não se vale de redes sequenciais como o RNN, então essa informação é passada através do embedding de posição.

Ex.

Position: [Eu] [amo] [NLP]

Embedding: [p0] [p1] [p2]

Somando esses três embeddings, o modelo consegue capturar semântica, posição e segmentação, resultando em uma representação rica da variável de entrada.

Ex.

O token final do “Eu” seria:

Token Embedding [v1] + Embedding de Posição [p0] + Embedding de Segmentação [sA]

Na figura 2 podemos ver os componentes da camada de encoder da arquitetura transformers.

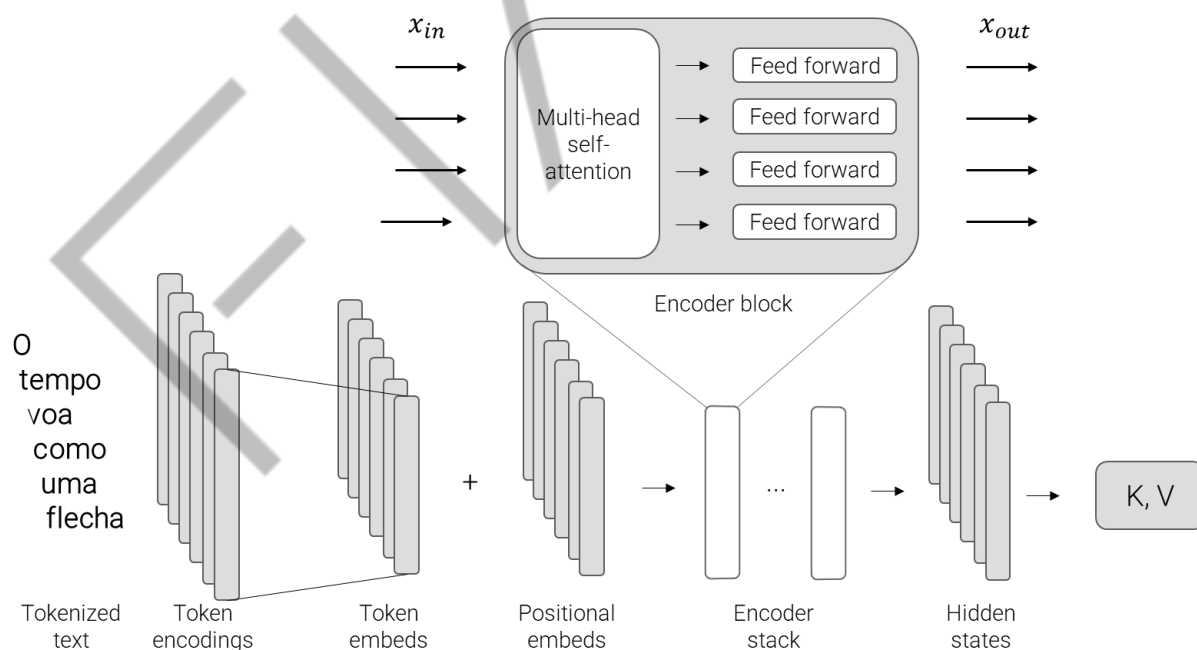


Figura 2 - Camada encoder da arquitetura transformers

Fonte: Tunstall et al. (2022)

O encoder da arquitetura transformers consiste em muitas camadas encoder empilhadas uma ao lado da outra. O principal objetivo delas é atualizar os embeddings

de entrada para produzir representações que contenham informação contextual da sequência. Conforme comentamos na aula anterior, isso é extremamente importante, visto que a mesma palavra pode ter significados diferentes dependendo do contexto.

Por exemplo, “Ele sempre traz um boné na cabeça” e “Rodrigo é o cabeça da equipe.”: a palavra “cabeça” na primeira frase se refere a uma parte do corpo humano, enquanto na segunda se refere à liderança. Dessa forma, “cabeça” estaria mais perto da palavra “liderança” se a palavra “equipe” estiver por perto.

No entanto, o bloco mais relevante e o que mais diferencia a arquitetura transformers diz respeito às chamadas camadas de atenção, que visam resolver a fraqueza das RNN. Cada camada do encoder recebe uma sequência de embeddings que alimenta uma camada de auto-atenção com muitas cabeças (multi-head self-attention) e uma camada feed forward totalmente conectada que é aplicada em cada entrada do embedding.

É a camada de self-attention que permite que as redes neurais deem pesos diferentes (atenções diferentes) para cada elemento na sequência. Para sequências de texto, elementos são tokens embeddings, em que cada token é mapeado para um vetor de dimensão fixa. Essa camada também é crucial para resolver outra fraqueza dos modelos recorrentes, que era a impossibilidade de paralelização, visto que são inerentemente sequenciais.

Nos transformers, os pesos das camadas de self-attention são computados para todos os hidden states em um mesmo set, ou seja, todos os hidden states do encoder. Isso faz com que os mecanismos de atenção associados aos modelos recorrentes envolvam computar a relevância de cada hidden state da camada de encoder para o hidden state da camada de decoder em uma determinada etapa de decodificação (decoding). Com isso, pode-se treinar modelos de uma forma muito mais rápida do que os modelos recorrentes.

A ideia principal por trás do self-attention é usar toda a sequência para calcular a média ponderada de cada embedding, em vez de passar embeddings fixos para cada token.

Anteriormente também comentei sobre multi-head self-attention. São necessárias mais de uma cabeça porque, quando aplicamos o softmax de uma única

cabeça, tendemos a focar em somente um aspecto de similaridade; quando temos mais de uma conseguimos focar em vários aspectos de similaridade de uma vez.

Por exemplo: uma cabeça pode focar na interação entre verbo e sujeito, enquanto outra procura por adjetivos próximos. No BERT temos um total de 12 cabeças de atenção, cada uma com dimensão 64 (768/12).

Não nos aprofundaremos mais nas camadas de atenção por hora, visto que vocês as estudarão amplamente no módulo de AI Generativa, e nem nas camadas de Decoder, que não são usadas no modelo BERT, mas serão usadas em outros modelos estudados também no módulo de AI Generativa.

A arquitetura que mencionamos até então é considerada o corpo do BERT. É uma arquitetura flexível, que retorna um hidden state para cada token. Agora, podemos incluir tarefas específicas adicionando as chamadas “cabeças”.

Dessa forma, o BERT é então pré-treinado em duas tarefas principais:

Masked Language Model (MLM): parte das palavras em uma frase são mascaradas aleatoriamente e o modelo é treinado para prever essas palavras. Isso força o modelo a aprender o contexto bidirecional. Exemplo: “Eu estudei muito a arquitetura [MASK] para aprender mais sobre [MASK].”

Next Sentence Prediction (NSP): o modelo é treinado para prever se uma sentença B segue uma sentença A, ajudando a aprender a relação entre frases.

Por causa da bidirecionalidade e do treinamento não supervisionado, o BERT pode ser treinado em tarefas específicas (fine tuning) usando somente uma camada adicional, sem mudanças de arquitetura substanciais.

As tarefas que podem ser realizadas são muitas, como: classificação de texto, reconhecimento de entidade nomeada (NER) e perguntas e respostas, entre outras.

Vantagens:

- **Contexto Bidirecional:** compreensão mais rica do contexto das palavras.
- **Versatilidade:** pode ser aplicado a diversas tarefas de PLN.
- **Desempenho:** supera muitos modelos anteriores em benchmarks de PLN.

Limitações:

- **Recursos Computacionais:** treinar e ajustar o BERT requer uma quantidade significativa de recursos computacionais.
- **Tamanho do Modelo:** os modelos BERT são grandes e podem ser difíceis de implantar em dispositivos com recursos limitados.

Vamos para exemplos práticos?

Para utilizá-lo, precisamos instalar as bibliotecas necessárias com o comando ``pip install transformers torch``.

Conforme comentamos anteriormente, o BERT foi treinado em texto na língua inglesa; no entanto, temos adaptações em várias outras, incluindo a língua portuguesa. Na língua portuguesa o mais famoso é o [BERTimbau](#) da Neural Mind AI, também disponível no [Hugging Face](#) e que foi treinado usando [brWaC Corpus](#).

Para a tarefa de análise de sentimento, passamos o texto e o sentimento resultante para treinar uma cabeça de classificação. A seguir, vamos experimentar um modelo já treinado com essa cabeça.

Exemplo de análise de sentimento

```
from transformers import BertTokenizer, BertForSequenceClassification, pipeline
model_name = 'neuralmind/bert-base-portuguese-cased'
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertForSequenceClassification.from_pretrained(model_name)
classifier = pipeline('sentiment-analysis', model=model, tokenizer=tokenizer)
text = "BERT é um modelo poderoso desenvolvido pelo Google para compreensão de linguagem natural."
result = classifier(text)
print(result)
```

Saída:

```
[{'label': 'LABEL_1', 'score': 0.5619189143180847}]
```

Você verá que, rodando esse código, aparecerá uma barra de progresso. Isso ocorre porque o pipeline automaticamente faz o download dos pesos do modelo. Isso

só acontecerá na primeira vez; nas demais a biblioteca verá que o download já foi feito e usará os pesos da versão em cache.

O resultado das previsões é uma lista de dicionários python e podemos utilizar a biblioteca pandas para apresentar de uma forma mais visual.

```
import pandas as pd
pd.DataFrame(result)
```

Note também que há retorno somente da classe mais provável para a análise de sentimento, visto que para inferir a outra classe basta fazer 1-score.

Aqui conseguimos usar o modelo sem a necessidade de fine tuning porque já foi realizado um treinamento na tarefa de análise de sentimento. No entanto, para outras tarefas talvez ele seja necessário.

Se você não quiser a versão em português, mas o BERT original, basta trocar a variável model_name para 'bert-base-uncased' e escrever seu texto em inglês. O "uncased" no nome diz se o modelo é sensível a letras maiúsculas ou minúsculas: o "uncased", no caso, é o não sensível. Viu como é simples?

Exemplo de perguntas e respostas:

```
from transformers import BertTokenizer, BertForQuestionAnswering
from transformers import pipeline
tokenizer = BertTokenizer.from_pretrained('bert-large-uncased-whole-word-masking-finetuned-squad')
model = BertForQuestionAnswering.from_pretrained('bert-large-uncased-whole-word-masking-finetuned-squad')
qa_pipeline = pipeline('question-answering', model=model, tokenizer=tokenizer)
context = "BERT is a model developed by Google for natural language understanding."
question = "Who developed BERT?"
result = qa_pipeline(question=question, context=context)
print(result)
```

Saída:

```
{'score': 0.9968146085739136, 'start': 29, 'end': 35, 'answer': 'Google'}
```

Para perguntas e respostas, a cabeça é treinada para prever a posição de início e a posição de fim de uma resposta para uma pergunta.

Exemplo de similaridade semântica:

```
import torch
from transformers import BertTokenizer, BertModel
from torch.nn.functional import cosine_similarity
model_name = 'neuralmind/bert-base-portuguese-cased' #
Para mudar para o inglês basta trocar por 'bert-base-uncased'.
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertModel.from_pretrained(model_name)
def encode_texts(texts):
    inputs = tokenizer(texts, padding=True, truncation=True,
return_tensors="pt")
    with torch.no_grad():
        outputs = model(**inputs)
        # Mean pooling of the token embeddings to get
sentence embeddings
        embeddings = outputs.last_hidden_state.mean(dim=1)
    return embeddings
texto1 = "BERT é um modelo poderoso desenvolvido pelo Google
para compreensão de linguagem natural."
texto2 = "Modelos de linguagem como BERT são usados para
várias tarefas de PLN."
# Encode dos textos
embeddings1 = encode_texts([texto1])
embeddings2 = encode_texts([texto2])
# Cálculo da similaridade de cossenos
similaridade = cosine_similarity(embeddings1, embeddings2)
print(f"Similaridade semântica: {similaridade.item()}")
```

Saída:

```
Similaridade semântica: 0.8233377933502197
```

Nesse exemplo, precisamos tokenizar, homogeneizar e limpar o texto. Você pode ver pela documentação do [BertTokenizer](#) que vários pré-processamentos já são realizados por padrão, como transformar o texto para letra minúscula, converter palavras não conhecidas para um token único indicando que é desconhecido, adicionar separação de sentenças e outros. Além disso, colocamos alguns outros parâmetros adicionais manualmente para garantir que os textos terão o mesmo tamanho.

Posteriormente, eles são transformados em representações vetoriais usando BERT. O `last_hidden_state` do BERT resulta nos embeddings para cada token no texto de entrada. Já o mean pooling dos embeddings é usado para computar um único vetor que representa toda a sentença. Por fim, é calculada a semelhança entre os vetores usando a similaridade de cossenos, que vai de -1, completamente diferentes, até 1, iguais.

É importante notar que há vários outros modelos que surgiram baseados no BERT que valem a pena ser mencionados, como o DistilBERT e o RoBERTa. O BERT, apesar de trazer excelentes resultados, é um modelo grande e difícil de implementar em produção em ambientes nos quais a baixa latência é necessária.

Durante o pré-treino, o DistilBERT usa uma técnica chamada de knowledge distillation que resulta em uma performance um pouco menor que o BERT, mas usando bem menos memória e muito mais rápido.

Já o RoBERTa por sua vez é treinado usando mais dados e somente na tarefa de Masked Language Model, não na Next Sentence Prediction. Isso evidenciou uma melhora na performance em comparação ao BERT original.

O QUE VOCÊ VIU NESTA AULA?

Nessa aula você aprendeu os componentes e como funciona a camada encoder da arquitetura transformers que é utilizada no modelo BERT.

Exploramos os modelos BERT e BERTimbau na prática, utilizando o Hugging Face Hub, para diversas tarefas como classificação de texto, perguntas e respostas e similaridade semântica.

Além disso, agora você já sabe de todo o potencial que existe nesse modelo para outras tarefas, como geração de texto, sumarização, tradução e muito mais.

REFERÊNCIAS

DEVLIN, J.; CHANG, M.; LEE, K.; TOUTANOVA, K. **BERT**: Pre-training of Deep Bidirectional Transformers for Language Understanding. 2019. Disponível em: <<https://arxiv.org/abs/1810.04805>>. Acesso em: 31 jul. 2024.

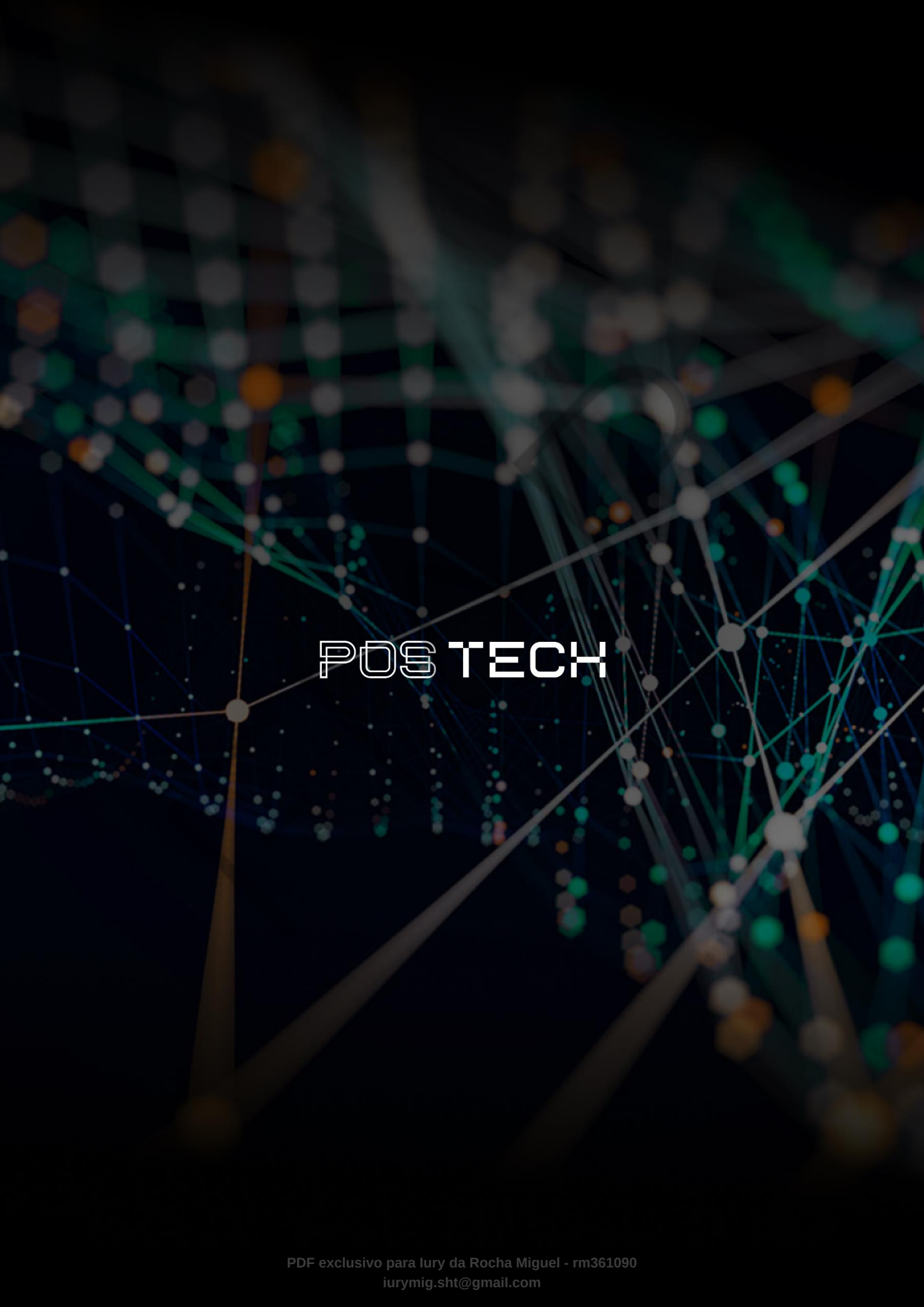
TUNSTALL, L.; VON WERRA, L.; WOLF, T. **Natural Language Processing with Transformers**. Revised Edition. [s.l.]: O'Reilly Media, Inc., 2022.

WAGNER, J. et al. **The brWaC Corpus**: A New Open Resource for Brazilian Portuguese. 2018. Disponível em: <https://www.researchgate.net/publication/326303825_The_brWaC_Corpus_A_New_Open_Resource_for_Brazilian_Portuguese>. Acesso em: 31 jul. 2024.

PALAVRAS-CHAVE

Processamento de Linguagem Natural. Transformers. Transferência de Conhecimento. BERT. AI Generativa. Aprendizado de Máquina.

EMENDAS

The background is a dark, abstract network visualization. It features a complex web of glowing nodes and connecting lines. The nodes are represented by small, semi-transparent spheres in various colors, including teal, orange, and grey. The lines are thin, light-colored threads that crisscross the entire frame, creating a sense of depth and connectivity. The overall effect is reminiscent of a digital or molecular structure.

POSTECH